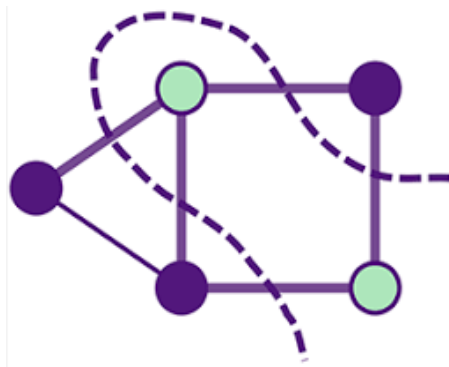


Quantum Approximate Optimization Algorithm(QAOA)

Definition

QAOA is a hybrid quantum-classical algorithm. A parameterized quantum circuit prepares a state, a classical optimizer tunes the circuit parameters to minimize a cost function, and the loop repeats until the parameters converge.

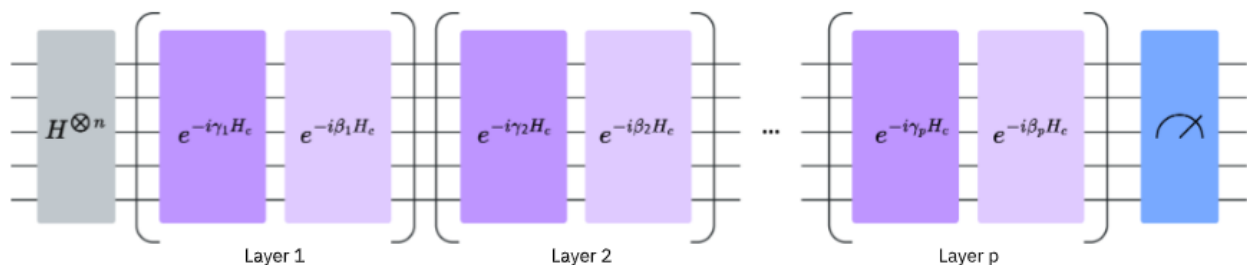
In this section , we will use QAOA to solve max - cut problem - an NP-hard optimization problem. Given a graph, split its nodes into two groups so that the number of edges between the two groups is maximized.



How QAOA works

1. Start in equal superposition of all possible solutions (apply H to every qubit)
2. Apply the cost operator — encodes the problem so that "good" solutions get a different phase than "bad" ones
3. Apply the mixer operator— spreads/mixes amplitudes across states, so the optimizer can explore
4. Measure → get a candidate solution
5. Classical optimizer adjusts angles (γ , β) based on how good the result was, and repeats

Steps 2–3 form one layer. Repeating p layers generally gives better results, at the cost of a deeper circuit.



In this section , we will solve max-cut with a small 3 node graph .

Setup

```
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
from scipy.optimize import minimize
import numpy as np
```

Initialize your problem by creating a graph with $n=3$ nodes.

```
n = 3
edge_list = [(0, 1), (1, 2), (0, 2)]
```

Build the circuit

```
def qaoa_circuit(gamma, beta):
    qc = QuantumCircuit(n, n)
    qc.h(range(n))

    # Cost Hamiltonian layer
    for (i, j) in edge_list:
        qc.cx(i, j)
        qc.rz(2 * gamma, j)
        qc.cx(i, j)

    # Mixer Hamiltonian layer
    for q in range(n):
        qc.rx(2 * beta, q)

    qc.measure(range(n), range(n))
    return qc

def cost_function(params):
    gamma, beta = params
    qc = qaoa_circuit(gamma, beta)
    sim = AerSimulator()
    counts = sim.run(qc, shots=1024).result().get_counts()

    total = 0
    for bitstring, count in counts.items():
        cut_value = 0
        for (i, j) in edge_list:
            if bitstring[i] != bitstring[j]:
                cut_value += 1
        total += cut_value * count

    avg_cut = total / 1024
    return -avg_cut
```

Use the classical optimizer

```
init_params = [0.5, 0.5]
result = minimize(cost_function, init_params, method='COBYLA')

print(f"Optimal gamma, beta : {result.x}")
print(f"Max average cut      : {-result.fun}")
```

Output :

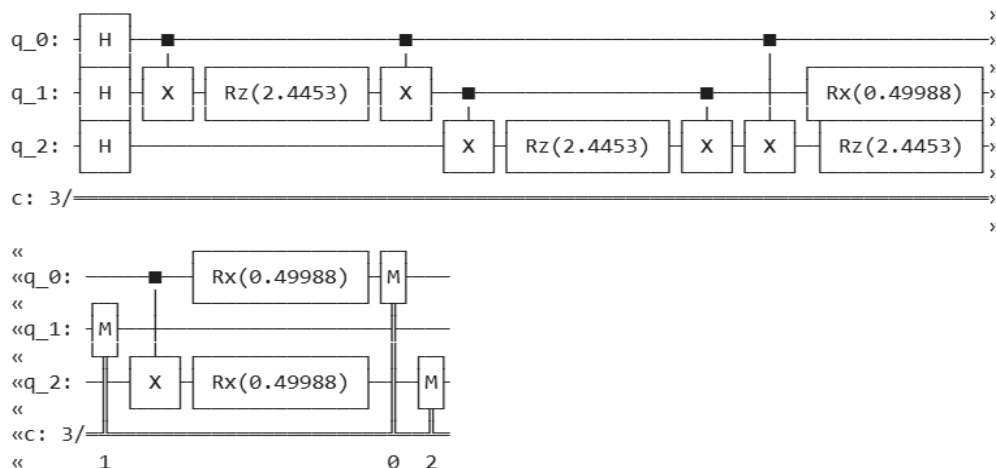
```
Optimal gamma, beta : [ 0.28385015 -0.32810252]
Max average cut      : 2.0
```

Run final circuit with optimal params

```
final_qc = qaoa_circuit(*result.x)
sim = AerSimulator()
counts = sim.run(final_qc, shots=1024).result().get_counts()
print(f"Final counts      : {counts}")
print(final_qc.draw('text'))
```

Output :

Final counts : {'110': 155, '000': 5, '010': 174, '111': 6, '011': 195, '001': 157, '101': 166, '100': 166}



Reading the output

The most frequent bitstring — 011 — represents the optimal Max-Cut partition: one node in one group, the other two in the second group, giving a cut value of 2.

Key takeaway

QAOA repeats a cost layer + mixer layer, with a classical optimizer tuning the angles γ and β in between. The cost Hamiltonian encodes *what* problem you're solving; the mixer Hamiltonian is generic and stays the same across problems. Only the cost Hamiltonian changes for Traveling Salesman, Portfolio Optimization, etc.